Clase 1: Condicionales, Funciones y Strings

Nicolás Alejandro Barnafi Wittwer
Pontificia Universidad Católica de Chile
Centro de Modelamiento Matemático

Pontificia Universidad Católica de Chile

Contents

1. Control de flujo
2. Funciones
3. Strings

Control de flujo

Ejemplo de salir de sala

Sintaxis bools: if/elif/else

```
d = 1 # Diámetro de figura
if figuraEsCuadrado: # resultado debe ser bool
    area = d**2 # INDENTACIÓN
elif figuraEsCirculo: # elif significa else if
    area = pi * (d/2)**2
else # llego acá si no pasa otra
    print("Error")
```

Consola

Contents

1. Control de flujo
2. Funciones
3. Strings

Funciones

Motivación

Supongamos queremos calcular IMC

```
# Enlistar algunos pesos y alturas
peso1 = 62.2
altura1 = 1.50
peso2 = 94.5
altura2 = 1.87

# Calcular IMC
imc1 = peso1 / altura1 / altura1 # p / h^2. Por qué?
imc2 = peso2 / altura2 / altura2
```

Sería mejor...

```
imc1 = calcularIMC(peso1, altura1)
imc2 = calcularIMC(peso2, altura2)
```

Motivación

Supongamos queremos calcular IMC

```
# Enlistar algunos pesos y alturas
peso1 = 62.2
altura1 = 1.50
peso2 = 94.5
altura2 = 1.87

# Calcular IMC
imc1 = peso1 / altura1 / altura1 # p / h^2. Por qué?
imc2 = peso2 / altura2 / altura2
```

Sería mejor...

```
imc1 = calcularIMC(peso1, altura1)
imc2 = calcularIMC(peso2, altura2)
```

Motivación

- Las funciones *reducen complejidad*
- Las funciones *vuelven código reutilizable*

Sintaxis

```
def functionName(arg1, arg2, ...):
    # [...]
    return expression # Retorno tiene tipo
    # Se puede omitir (return None)
```

Por ejemplo, en el caso del IMC

```
def calcularIMC(peso, altura):
    return peso / altura**2
```

Ojo con conflictos de nombre (dentro y fuera de función)

La función `print`

Esta función muestra el valor de una variable.

- Archivo de ejemplo 'test.py':

```
a = 2
b = 3*a
print(b)
```

- Lo que vemos al ejecutarlo:

```
$ python test.py
$ 6
```

Veamos algunos ejemplos en consola

Argumentos por keyword

Podemos definir argumentos *por defecto*

```
def printName(nombre="Nicolás", apellido="Barnafi"):
    print("Me llamo ", nombre, " ", apellido)

printName("nicolás", "barnafi") # en orden
printName("nicolás")            # 1 por defecto
printName()                     # 2 por defecto
printName(nombre="nico")        # keyword arg
printName(apellido="bfi")       # keyword arg
```

Sintaxis mal documentada

```
def printName(nombre="Nicolás", apellido="Barnafi"):
    print("Me llamo ", nombre, " ", apellido)

lista = ["nico", "barnafi"]
diccionario = {"nombre": "nico",
               "apellido": "barnafi"}
printName(*lista) # por lista
printName(**kwargs) # por diccionario
```

Ejemplo

Veamos cómo calcular el área de círculos

$$A(r) = \pi r^2$$

Qué pasa con los argumentos equivocados?

Type-checking: `type(a) == int`

Ejemplo

Veamos cómo calcular el área de círculos

$$A(r) = \pi r^2$$

Qué pasa con los argumentos equivocados?

Type-checking: `type(a) == int`

Scope o alcance de nombres

Qué pasa en este código?

```
a = 2
def f(a):
    b = a**2
    return b
print(f(3))
```

Contents

1. Control de flujo
2. Funciones
3. Strings

Strings

Definición

- Un 'string' es un tipo de dato que contiene texto.

```
> s = "hola :)"
> print(s) # Imprime 'hola :)'
```

- Está compuesto por caracteres, se pueden acceder por componentes:

```
> s = "hola :)"
> print(s[0], s[1], s[2]) # Imprime 'hol'
```

- Tienen MUCHAS funcionalidades

Definición

- Un 'string' es un tipo de dato que contiene texto.

```
> s = "hola :)"
> print(s) # Imprime 'hola :)'
```

- Está compuesto por caracteres, se pueden acceder por componentes:

```
> s = "hola :)"
> print(s[0], s[1], s[2]) # Imprime 'hol'
```

- Tienen MUCHAS funcionalidades

Definición

- Un 'string' es un tipo de dato que contiene texto.

```
> s = "hola :)"
> print(s) # Imprime 'hola :)'
```

- Está compuesto por caracteres, se pueden acceder por componentes:

```
> s = "hola :)"
> print(s[0], s[1], s[2]) # Imprime 'hol'
```

- Tienen MUCHAS funcionalidades

Functiones del objeto

Los strings, como muchos otros objetos, tienen funciones.
Si tenemos `s = "hola string"`:

- `s.upper()`: Nuevo string 'HOLA STRING'
- `s.replace('str', 'k')`: Nuevo string 'hola king'
- `s.find('a')`: Entrega índice '3'.
- Hay MUCHAS (ver `dir(s)`)

En todas ellas, se general un `str` nuevo, el original queda intacto.
Cuándo podría ser un problema esto?

La librería `string`

Usaremos nuestra primera librería!

- Las librerías agregan funcionalidades específicas
- Sintaxis para importar librerías:

```
> import string
```

- Esto genera una variable del nombre de la librería con funciones

Veamos cómo se puede usar

La librería `string`

Usaremos nuestra primera librería!

- Las librerías agregan funcionalidades específicas
- Sintaxis para importar librerías:

```
> import string
```

- Esto genera una variable del nombre de la librería con funciones

Veamos cómo se puede usar

El método `format`

$$\texttt{algo} \underbrace{\texttt{.}}_{\text{llamar función}} \mathit{method}()$$

Sirve para imprimir variables:

```
> a = 20.17
> "a es {}".format(a) # imprime 20.17
> f"a es {a}" # imprime 20.17
> f"a es {a:2.1f}" # imprime 20.1
> f"a es {a:2.2e}" # imprime 2.02e+01
```

Método de objeto $\neq$ método de tipo

Recap

- Control de flujo (`if`/`elif`/`else`)
- Cómo definir funciones
- Vimos el alcance (scope) de variables
- Los strings son tipos de datos con funciones
- La librería 'string'

Clase 1: Condicionales, Funciones y Strings

Nicolás Alejandro Barnafi Wittwer
Pontificia Universidad Católica de Chile
Centro de Modelamiento Matemático

Pontificia Universidad Católica de Chile

Mini ejercicios

1. Implemente y ejecute todos los códigos vistos en clases
2. Implemente una función que diga si un número es par. Podrá servir lo siguiente:

```
a = 100
a % 2 # Resto de dividir por 2
```